

TERRAFORM

INFRASTRUCTURE AS CODE



Write once.
Deploy
anywhere.

Define • Provision • Manage • Scale



Declarative

Write infrastructure
in HCL



Consistent

Versioned, repeatable
& reliable



Safe

Plan before apply,
reduce human errors



Automated

Integrate with CI/CD
for full automation



PROVIDERS

Connect to any
cloud or platform



STATE

Track real
infrastructure



BACKEND

Store state
securely



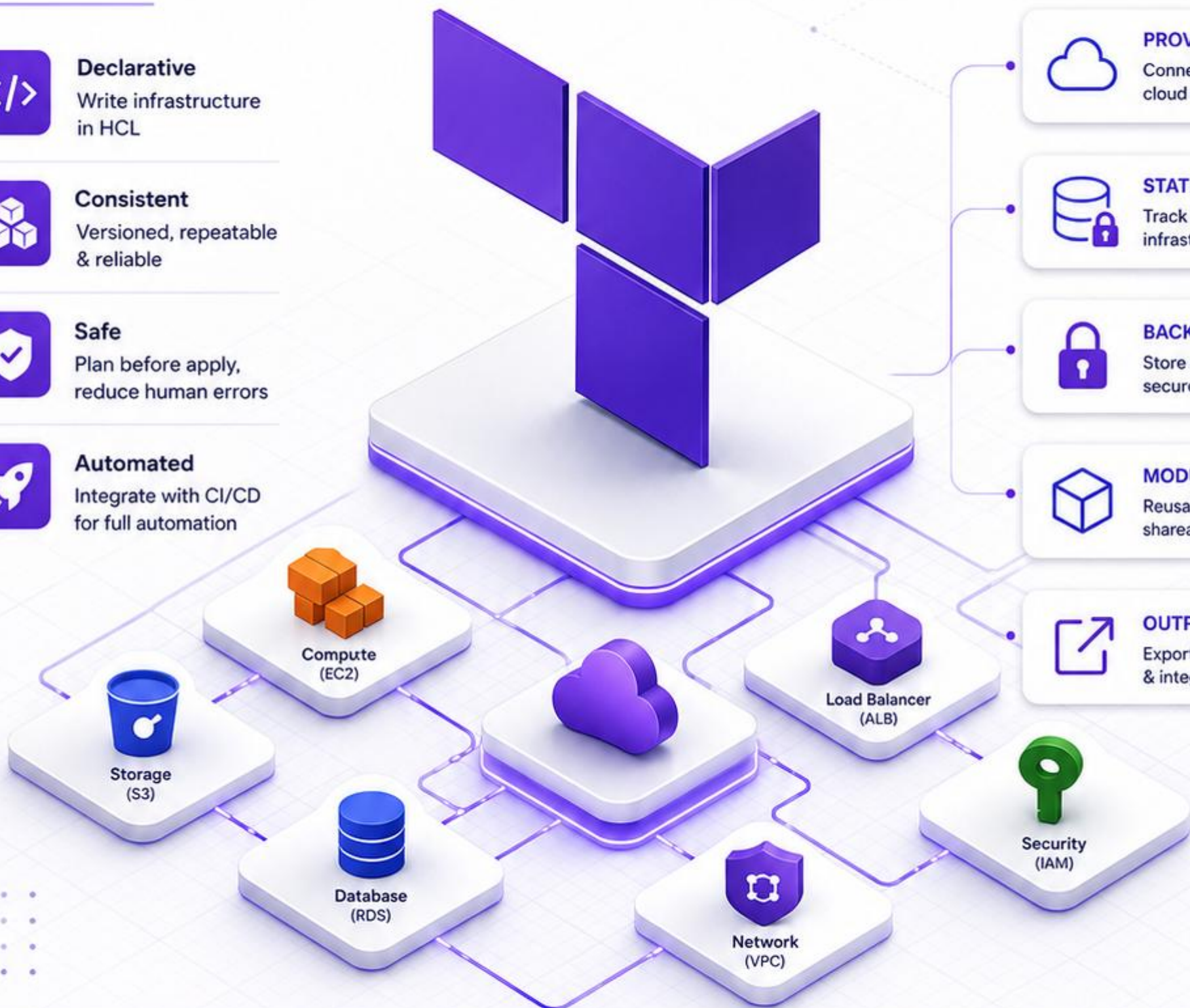
MODULES

Reusable &
shareable code



OUTPUTS

Export values
& integrate



Infrastructure
as Code



Idempotent &
Predictable



Faster
Provisioning



Versioned &
Collaborative



Secure &
Auditable



[linkedin.com/in/siranjeevi1](https://www.linkedin.com/in/siranjeevi1)



siranjeevis.netlify.app

TERRAFORM ARCHITECTURE

Infrastructure as Code, built for automation, consistency & scale.



DECLARATIVE

Define the desired state of infrastructure



AUTOMATED

Provision, update & manage resources



CONSISTENT

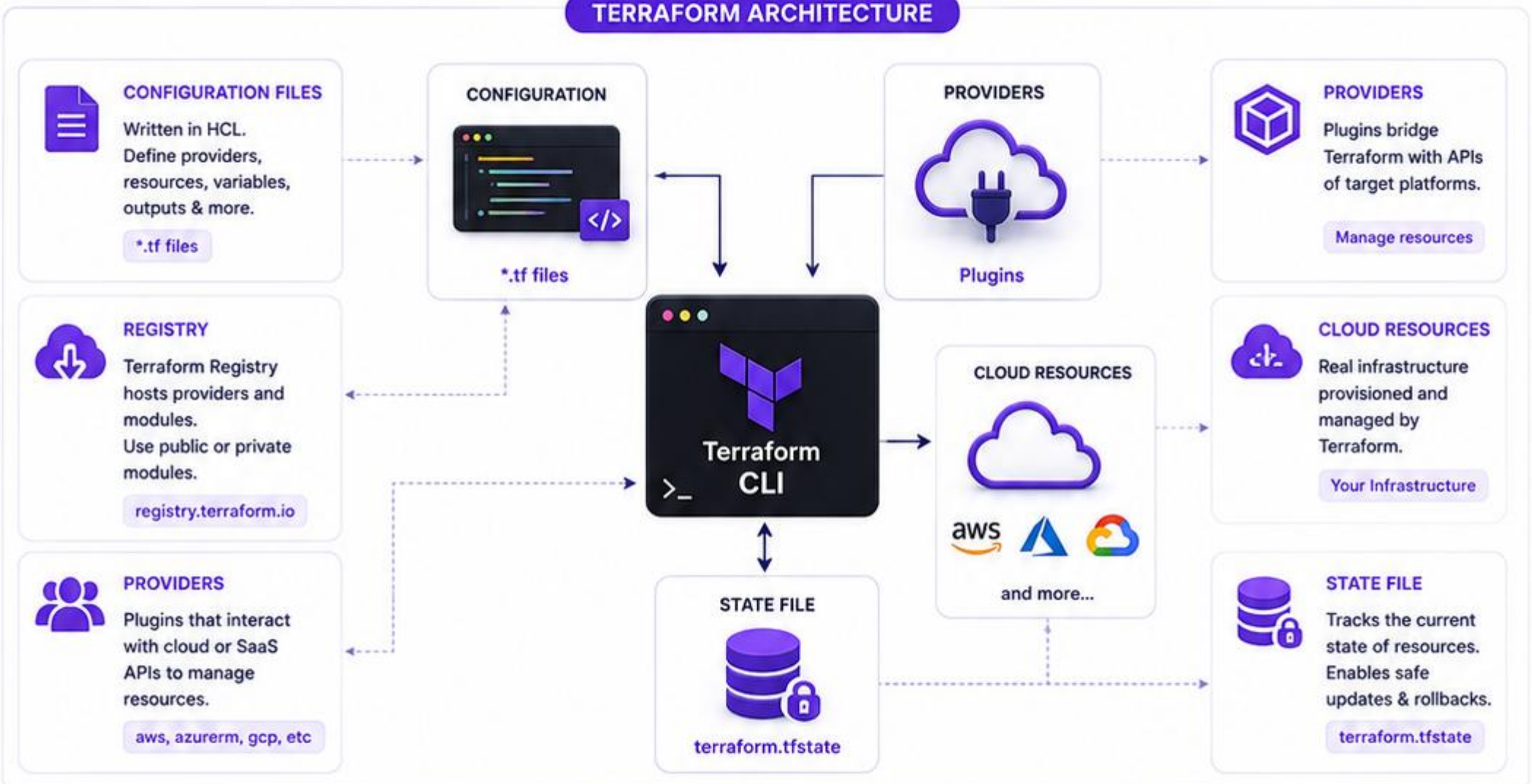
Versioned, repeatable & reliable deployments



SCALABLE

Manage infrastructure at any scale

TERRAFORM ARCHITECTURE



TERRAFORM WORKFLOW



BEST PRACTICES



Use Remote State with locking



Version Control your code



Review Plan before Apply



Use Modules for reusability



Secure Secrets and credentials



linkedin.com/in/siranjeevi1



siranjeevis.netlify.app

TERRAFORM CONFIGURATION

Building blocks to define, manage and provision infrastructure.



DECLARATIVE

Define desired state using HCL.



REUSABLE

Use modules to reuse configuration.



CONSISTENT

Versioned, predictable & easy to maintain.



AUTOMATED

Provision & manage infrastructure safely.

TERRAFORM CONFIGURATION BLOCKS



1. TERRAFORM { }

Defines Terraform settings, required version and backend configuration.

```
terraform { ... }
```



2. PROVIDER

Configures the cloud provider to interact with its APIs.

```
provider "aws" { ... }
```



3. RESOURCE

Defines the infrastructure objects to be created or managed.

```
resource "aws_instance" "web" { ... }
```

123

4. VARIABLE

Defines input variables to make configuration dynamic and reusable.

```
variable "instance_type" { ... }
```

```
terraform {
  required_version = ">= 1.5.0"
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}

provider "aws" {
  region = var.aws_region
}

variable "aws_region" {
  description = "AWS region to deploy resources"
  type        = string
  default     = "us-east-1"
}

locals {
  name_prefix = "tf-demo"
  common_tags = {
    Project = "Terraform"
    Environment = "Dev"
  }
}

resource "aws_instance" "web" {
  ami           = data.aws_ami.ubuntu.id
  instance_type = var.instance_type
  tags = merge(local.common_tags, {
    Name = "${local.name_prefix}-web"
  })
}

data "aws_ami" "ubuntu" {
  most_recent = true
  owners      = ["099720109477"]
  filter {
    name   = "name"
    values = ["ubuntu/images/hvm-ssd/ubuntu-jammy-22.04-amd64-server-*"]
  }
}

output "instance_public_ip" {
  description = "Public IP of the EC2 instance"
  value       = aws_instance.web.public_ip
}

module "vpc" {
  source = "terraform-aws-modules/vpc/aws"
  name   = "${local.name_prefix}-vpc"
  cidr   = "10.0.0.0/16"
}
```



5. LOCALS

Assigns names to expressions for reuse within the configuration.

```
locals { ... }
```



6. DATA SOURCE

Fetches information from existing resources or external systems.

```
data "aws_ami" "ubuntu" { ... }
```



7. OUTPUT

Exposes values from the configuration after provisioning.

```
output "instance_public_ip" { ... }
```



8. MODULE

Groups multiple resources into a reusable, shareable component.

```
module "vpc" { ... }
```

BEST PRACTICES



Use variables for flexibility, not hardcoding.



Store state remotely with locking.



Use consistent tagging across resources.



Validate & plan before applying changes.



Prefer modules for reusability and scale.



linkedin.com/in/siranjeevi1



siranjeevis.netlify.app

TERRAFORM STATE

Keep track of your infrastructure. Enable collaboration & safe changes.



SOURCE OF TRUTH

State maps real infrastructure to your Terraform config.



COLLABORATION

Remote state with locking enables safe team collaboration.



RELIABLE

Enables consistent plans, updates & rollbacks.



RECOVERABLE

State backup helps restore infrastructure when needed.

1. WHAT IS TERRAFORM STATE?

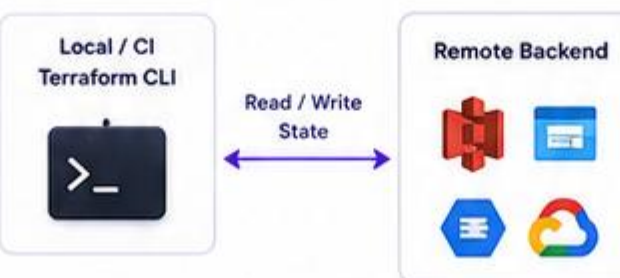
State is a JSON file that stores the mapping between your Terraform configuration and the real infrastructure.

```
1 {
2   "version": 4,
3   "terraform_version": "1.5.0",
4   "serial": 15,
5   "lineage": "d2b1c7a5-....",
6   "resources": [
7     {
8       "mode": "managed",
9       "type": "aws_instance",
10      "name": "web",
11      "instances": [
12        {
13          "attributes": {
14            "id": "i-0123456789abcdef0",
15            "instance_type": "t3.micro",
16            "ami": "ami-0abc123456789def"
17          }
18        }
19      ]
20    }
21  ]
22 }
```

Default location: ./terraform.tfstate

2. REMOTE STATE

Store state remotely to enable team collaboration and improve security.



- Centralized state storage
Team members share the same state
- Better security & access control
Restrict access using IAM / RBAC
- State versioning & recovery
Enable backup, restore & audit

Common backends: S3, Azure Storage, GCS

3. BACKEND CONFIGURATION

Backend defines where and how Terraform stores the state file.

```
terraform {
  backend "s3" {
    bucket = "my-tf-state-bucket"
    key    = "envs/prod/terraform.tfstate"
    region = "us-east-1"
    dynamodb_table = "tf-state-lock"
    encrypt   = true
  }
}
```

- bucket / container
Remote storage location
- key
Path to the state file
- region
Cloud region
- dynamodb_table
Table used for state locking
- encrypt
Encrypt state at rest

4. STATE LOCKING

Locking prevents multiple users from modifying state simultaneously.



- Ensures only one operation at a time
- Prevents state corruption
- Enabled using DynamoDB table (AWS)

5. STATE COMMANDS

Useful commands to inspect and manage state.

<code>terraform state list</code>	List resources in state
<code>terraform state show <addr></code>	Show details of a resource
<code>terraform state mv <src> <dest></code>	Move/rename resource in state
<code>terraform state rm <addr></code>	Remove resource from state
<code>terraform import <addr> <id></code>	Import existing resource to state
<code>terraform state pull</code>	Download state to stdout
<code>terraform state push</code>	Push local state to backend
<code>terraform state replace-provider</code>	Replace provider in state

6. BEST PRACTICES

- Use Remote Backend
Store state remotely (S3, Azure, GCS).
- Enable State Locking
Use DynamoDB / Blob lease for locking.
- Version & Backup State
Enable versioning and regular backups.
- Restrict Access
Use IAM / RBAC to control access.
- Never Edit State Manually
Use Terraform CLI to modify state.
- Separate State per Environment
Keep dev, stage, prod states isolated.
- Monitor & Audit
Track who changed what and when.

STATE FILE LIFECYCLE



REMEMBER

Your infrastructure is only as reliable as your state management.



linkedin.com/in/siranjeevi1



siranjeevis.netlify.app

TERRAFORM PROVIDERS & RESOURCES

Connect to cloud. Define infrastructure. Manage dependencies.



CONNECT

Providers connect Terraform to APIs.



PROVISION

Resources define the infrastructure.



AUTOMATE

Dependencies are handled automatically.



MANAGE

Meta-arguments & provisioners add control.

1. PROVIDERS

Providers are plugins that allow Terraform to interact with cloud/platform APIs.

```
terraform {
  required_providers {
    aws = {
      source = "hashicorp/aws"
      version = "~> 5.0"
    }
  }

  provider "aws" {
    region = "us-east-1"
  }
}
```

Popular Providers



...and many more

Providers authenticate & talk to APIs to create and manage resources.

2. RESOURCES

Resources are the infrastructure objects managed by Terraform.

```
resource "aws_instance" "web" {
  ami           = "ami-0abc123456789def0"
  instance_type = "t3.micro"

  tags = {
    Name        = "terraform-web"
    Environment = "dev"
  }
}
```



EC2 Instance



S3 Bucket



RDS Database



VPC Network

...and many more resources

Terraform tracks the desired state of resources in the state file.

3. DATA SOURCES

Data sources allow you to fetch information from existing resources.

```
data "aws_ami" "ubuntu" {
  most_recent = true
  owners      = ["099720109477"]

  filter {
    name   = "name"
    values = ["ubuntu/images/hvm-ssd/ubuntu-jammy-22.04-amd64-server-*"]
  }
}
```

Fetch existing infrastructure details

Use outputs in other resources

Read-only, does not create anything

Useful for dynamic values like AMI IDs, VPC IDs, subnet IDs, etc.

4. DEPENDENCIES

Terraform builds a dependency graph and creates resources in the right order.

Implicit Dependencies (Automatic)



Explicit Dependencies (Using depends_on)

```
resource "aws_instance" "app" {
  ami           = "ami-123456"
  instance_type = "t3.micro"

  depends_on = [
    aws_db_instance.db
  ]
}
```

- Implicit: Reference creates automatic dependency
- Explicit: Use depends_on when needed
- Ensures correct creation & destruction order

Terraform handles the dependency graph for you.

5. META-ARGUMENTS

Meta-arguments change how resources behave.

Meta-Argument	Purpose	Example
count	Create multiple instances	count = 3
for_each	Create based on a map or set	for_each = var.subnets
depends_on	Explicit dependency on other resources	depends_on = [aws_iam_role.r]
lifecycle	Control resource lifecycle	prevent_destroy = true
provider	Use alternate provider	provider = aws.usw2
aliases	Configure multiple providers	alias = "virginia"

Meta-arguments give you more control and flexibility.

6. PROVISIONERS

Provisioners run scripts or commands on local or remote machines.

Types

- local-exec**
Run commands on local machine
- remote-exec**
Run commands on the resource
- file**
Copy files to the resource

Example (remote-exec)

```
resource "aws_instance" "web" {
  ami           = "ami-123456"
  instance_type = "t3.micro"

  provisioner "remote-exec" {
    inline = [
      "sudo apt update -y",
      "sudo apt install -y nginx",
      "sudo systemctl start nginx"
    ]
  }
}
```

- Use for configuration, bootstrap, setup
- Not for managing state - use config mgmt tools
- Prefer idempotent scripts

Use sparingly. Prefer automation tools like Ansible.

BEST PRACTICES



Pin provider versions for consistency



Use least privilege IAM for providers



Tag all resources



Review plan before applying changes



Use data sources instead of hardcoding values

PROVIDER EXAMPLE (AWS)

```
provider "aws" {
  region = "us-east-1"
  profile = "dev"
}
```



linkedin.com/in/siranjeevi1



siranjeevis.netlify.app

TERRAFORM MODULES

Build reusable. Share smart. Deploy anywhere.

REUSE
Write once,
use everywhere.

STANDARDIZE
Maintain consistency
across environments.

SCALE
Build complex
infrastructure
with simplicity.

COLLABORATE
Share modules
across teams.



REUSABLE
Create once and reuse
across projects.



MAINTAINABLE
Easy to update, version
and manage.



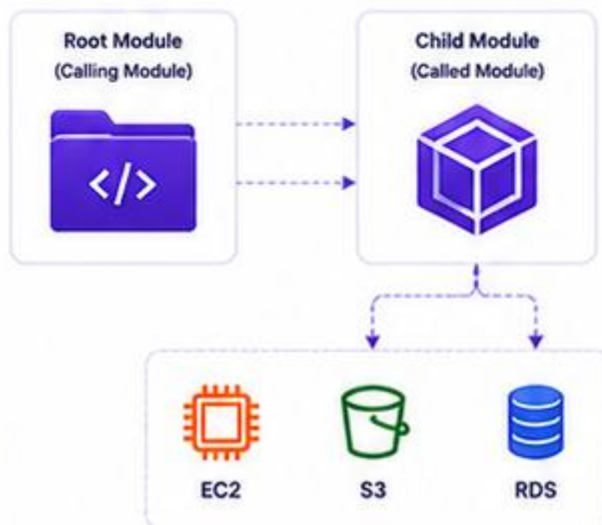
CONSISTENT
Enforce best practices
and standards.



FASTER DELIVERY
Accelerate infrastructure
delivery with modules.

1. MODULE OVERVIEW

Modules are containers for multiple resources that are used together.



i Root module calls child modules.
Child modules contain resources and logic.

2. MODULE STRUCTURE

```
my-infrastructure/
|-- main.tf           # Root module
|-- variables.tf      # Input variables
|-- outputs.tf        # Output values
|-- terraform.tfvars  # Variable values
|-- modules/         # Child module
    |-- vpc/
        |-- main.tf
        |-- variables.tf
        |-- outputs.tf
```

main.tf	Resource definitions and module calls
variables.tf	Define input variables
outputs.tf	Define output values
terraform.tfvars	Provide variable values
modules/	Directory for reusable modules

3. MODULE INPUTS (VARIABLES)

Pass inputs to customize module behavior.

```
module "vpc" {
  source      = "../modules/vpc"
  cidr_block  = "10.0.0.0/16"
  azs         = ["us-east-1a", "us-east-1b"]
  enable_nat_gateway = true
  tags = {
    Environment = "dev"
    Project      = "terraform-modules"
  }
}
```

Example variables in child module:

```
variable "cidr_block" {
  type      = string
  description = "VPC CIDR block"
}

variable "azs" {
  type      = list(string)
  description = "Availability Zones"
}

variable "enable_nat_gateway" {
  type      = bool
  default   = false
}
```

4. MODULE OUTPUTS

Return values from modules to use in root module.

```
output "vpc_id" {
  description = "ID of the VPC"
  value       = aws_vpc.this.id
}

output "public_subnet_ids" {
  description = "List of public subnet IDs"
  value       = aws_subnet.public[*].id
}
```

Used in root module:
module.vpc.vpc_id
module.vpc.public_subnet_ids

i Outputs make modules composable.

5. MODULE REGISTRY

Publish and discover modules in the Terraform Registry.



registry.terraform.io
The official hub for sharing and using modules.

6. REUSABILITY & VERSIONING

Version your modules for safe and reliable reuse.

```
module "vpc" {
  source      = "terraform-aws-modules/vpc/aws"
  version     = "5.1.0" # Pin to a specific version

  cidr      = "10.0.0.0/16"
  azs       = ["us-east-1a", "us-east-1b"]
}
```

- ✓ Pin module versions to ensure stability.
- ✓ Use semantic versioning (MAJOR.MINOR.PATCH).
- ✓ Update versions after testing in lower environments.
- ✓ Avoid breaking changes in your modules.

BEST PRACTICES



Keep modules
small and focused.



Use clear
inputs and
outputs.



Document variables,
outputs and usage
clearly.



Version your
modules and pin
versions.



Publish modules
for team and
community use.



Test modules
before
releasing.



Avoid hardcoding.
Make modules
flexible.



linkedin.com/in/siranjeevi1



siranjeevis.netlify.app

TERRAFORM WORKFLOW

From code to cloud. A consistent workflow for reliable infrastructure.



PREDICTABLE

Plan, preview & approve before changes.



CONSISTENT

Same workflow across environments.



SAFE

Validate, review & apply with confidence.



AUTOMATED

Easy to integrate with CI/CD pipelines.



REPEATABLE

Destroy & recreate without manual effort.

TERRAFORM CLI WORKFLOW

1 INIT

Initialize the working directory and download providers & modules.

```
$ terraform init
```



- Downloads providers
- Initializes backend
- Sets up modules

2 VALIDATE

Validate the syntax and configuration files.

```
$ terraform validate
```



- Checks syntax
- Validates configuration
- Ensures correctness

3 FMT

Reformat configuration files to standard style.

```
$ terraform fmt
```



- Consistent formatting
- Improves readability
- Follows best practices

4 PLAN

Create an execution plan and preview changes.

```
$ terraform plan
```



- Shows changes
- Highlights additions, updates & deletes
- Review before apply

5 APPLY

Apply the changes to reach desired state.

```
$ terraform apply
```



- Provisions resources
- Updates infrastructure
- Saves state file

6 DESTROY

Destroy resources and clean up.

```
$ terraform destroy
```



- Destroys resources
- Cleans up infrastructure
- Removes state

i Always review the plan before applying. Use version control and remote state for team collaboration.

PRODUCTION DEPLOYMENT FLOW

1. CODE

Write Terraform configuration



.tf files

2. VERSION CONTROL

Commit & push to repository



GitHub / GitLab / Bitbucket

3. CI PIPELINE

Run quality & security checks



Lint • Validate • Test • Security

4. PLAN & APPROVAL

Generate plan and get approval

```
terraform plan
No changes. /
+ 2 to add
```



Manual Approval / Policy Check

5. APPLY

Apply changes to infrastructure



terraform apply

6. MONITOR

Monitor & verify infrastructure



CloudWatch / Monitor

Feedback loop for continuous improvements

REMOTE STATE (PRODUCTION)

Terraform CLI



Read / Write

Remote Backend



Lock / Unlock

State Lock



DynamoDB / Blob Lease

- ✓ Store state in remote backend (S3, Azure Blob, GCS)
- ✓ Enable state locking to prevent concurrent changes

RECOMMENDED BACKEND SETUP (AWS)

```
terraform {
  backend "s3" {
    bucket           = "my-tf-state-bucket"
    key              = "envs/prod/terraform.tfstate"
    region           = "us-east-1"
    dynamodb_table  = "tf-state-lock"
    encrypt           = true
  }
}
```

- Versioning enabled on S3
- DynamoDB for state locking

TIPS

- ✓ Use remote backend for state.
- ✓ Enable state locking.
- ✓ Review plan before apply.
- ✓ Use workspaces for isolation.
- ✓ Backup state regularly.
- ✓ Use least privilege IAM.
- ✓ Automate with CI/CD.



linkedin.com/in/siranjeevi1



siranjeevis.netlify.app

TERRAFORM BEST PRACTICES & SECURITY

Build secure, scalable and maintainable infrastructure.



SECURE

Follow security best practices.



RELIABLE

Ensure consistency and reduce risks.



COLLABORATIVE

Enable safe teamwork with remote state.



EFFICIENT

Automate workflows and save time.



SCALABLE

Design for growth and repeatability.

1. REMOTE BACKEND

Store state remotely for durability, collaboration and access control.

```
terraform {
  backend "s3" {
    bucket = "my-tf-state-bucket"
    key    = "envs/prod/terraform.tfstate"
    region = "us-east-1"
    dynamodb_table = "tf-state-lock"
    encrypt = true
  }
}
```



AWS S3



Azure Blob



Google Cloud Storage

- ✓ Centralized and secure state storage
- ✓ Enable versioning for state file
- ✓ Restrict bucket access using IAM

2. SECRETS MANAGEMENT

Never hardcode secrets in code or state files.



AWS
Secrets Manager



Azure
Key Vault



Google
Secret Manager

```
resource "aws_db_instance" "example" {
  identifier = "prod-db"
  engine    = "mysql"
  username  = "admin"
  password  = data.aws_secretsmanager_secret_version.dbpass.secret_string
}

data "aws_secretsmanager_secret_version" "dbpass" {
  secret_id = "prod/db/password"
}
```

- ✓ Use managed secret stores (AWS, Azure, GCP)
- ✓ Reference secrets via data sources
- ✓ Limit secret access with least privilege
- ✓ Never commit secrets to VCS

3. STATE LOCKING

Prevent concurrent changes and state corruption.



Team



State Lock
(DynamoDB / Blob Lease)



Remote State

```
resource "aws_dynamodb_table" "tf_lock" {
  name           = "tf-state-lock"
  billing_mode   = "PAY_PER_REQUEST"
  hash_key       = "LockID"
  attribute {
    name = "LockID"
    type = "S"
  }
}
```

- ✓ Use DynamoDB (AWS) or Blob Lease (Azure) for locking
- ✓ Ensures only one operation at a time
- ✓ Prevents state file corruption

4. VERSION PINNING

Lock provider and module versions for consistency.

Provider Version Pinning

```
terraform {
  required_providers {
    aws = {
      source = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}
```



Avoid unexpected breaking changes

Module Version Pinning

```
module "vpc" {
  source = "terraform-aws-modules/vpc/aws"
  version = "~> 5.1"
  # ...
}
```

- ✓ Pin provider versions using required_providers
- ✓ Pin module versions using version argument
- ✓ Test updates in lower environments first

5. WORKSPACES

Manage multiple environments using workspaces.

```
$ terraform workspace new prod
$ terraform workspace select prod
$ terraform workspace list
```



Configuration
(Shared)

Workspaces



dev



staging



prod

- ✓ Use workspaces for dev, staging, prod isolation
- ✓ Keep state files separate per workspace
- ✓ Combine with remote backend for best results

Workspaces are not a security boundary.

6. CI/CD INTEGRATION

Automate Terraform workflows with CI/CD.



Code
Commit



CI Pipeline
(Test)



Terraform
Plan



Approval
(Gate)



Terraform
Apply

Example: GitHub Actions (Plan)

```
- name: Terraform Plan
  run: |
    terraform init
    terraform validate
    terraform plan -out=tfplan
- name: Upload Plan
  uses: actions/upload-artifact@v4
  with:
    name: tfplan
    path: tfplan
```

- ✓ Run init, validate, plan in every pipeline
- ✓ Require manual approval for apply
- ✓ Store tf plan as artifact for audit
- ✓ Use OIDC / IRSA for secure cloud auth

ADDITIONAL BEST PRACTICES



Use least privilege
IAM / RBAC
permissions



Keep modules
small, focused &
reusable



Review plans
before applying
changes



Backup state
regularly and
test restore



Tag resources
to track cost
and ownership



Security is not optional. Build it into your Terraform workflows from day one.



linkedin.com/in/siranjeevi1



siranjeevis.netlify.app

TERRAFORM ADVANCED CHEAT SHEET

Advanced topics overview for power users.



POWERFUL

Advanced features for complex needs.



CONTROL

Full control over state, resources & workflows.



DEBUG

Troubleshoot and solve issues faster.



AUTOMATE

Integrate with CI/CD and automation.



RELIABLE

Follow best practices for production.

1 CLI COMMANDS



- Core CLI commands to manage infrastructure.
- Validate, format, plan, apply and destroy.
- Essential for day-to-day Terraform operations.



The foundation of every Terraform workflow.

2 STATE COMMANDS



- Manage and inspect Terraform state safely.
- List, show, pull, push, backup and modify state.
- Critical for maintaining state integrity.



Protect and control your infrastructure state.

3 WORKSPACE COMMANDS



- Manage multiple environments in a single configuration.
- Create, switch, list and delete workspaces.
- Great for dev, staging, and production isolation.



One configuration, multiple environments.

4 IMPORT



- Bring existing infrastructure under Terraform management.
- Import resources to track and manage with Terraform.
- First step towards full infrastructure as code.



Adopt existing, manage with confidence.

5 TAIN



- Mark resources for recreation on next apply.
- Useful for replacing unhealthy or misconfigured resources.
- Helps recover without manual intervention.



Force rebuild when needed.

6 CONSOLE



- Interactive shell to evaluate expressions.
- Test variables, outputs and functions.
- Powerful for debugging and exploration.



Test and inspect in real-time.

7 DEBUGGING



- Debug logs and detailed output for troubleshooting.
- Use log levels and trace logs for deep insights.
- Identify provider, state and configuration issues.



Find issues faster, fix with precision.

8 COMMON ERRORS



- State lock issues and how to resolve.
- Provider/version mismatch problems.
- Dependency and permission related errors.



Understand, avoid and resolve quickly.



[linkedin.com/in/siranjeevi1](https://www.linkedin.com/in/siranjeevi1)



siranjeevis.netlify.app